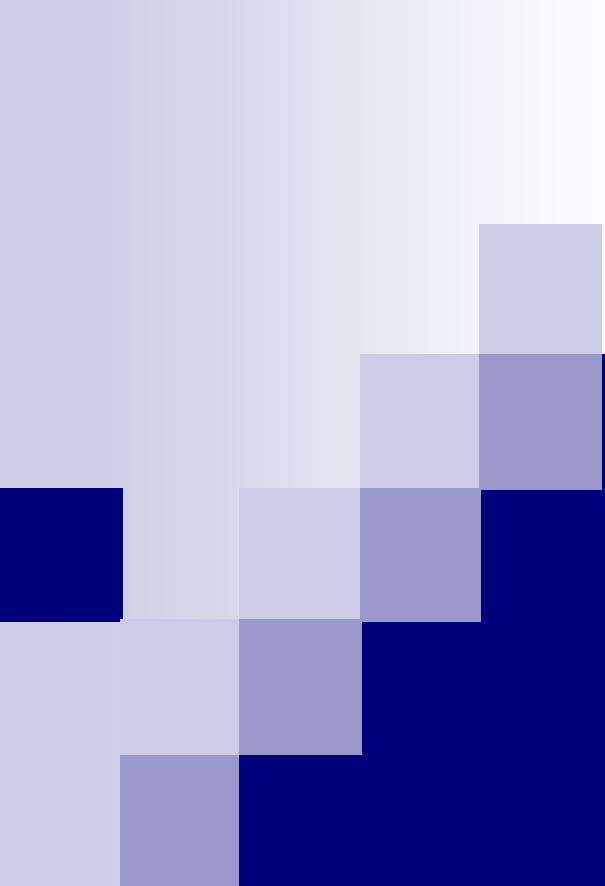




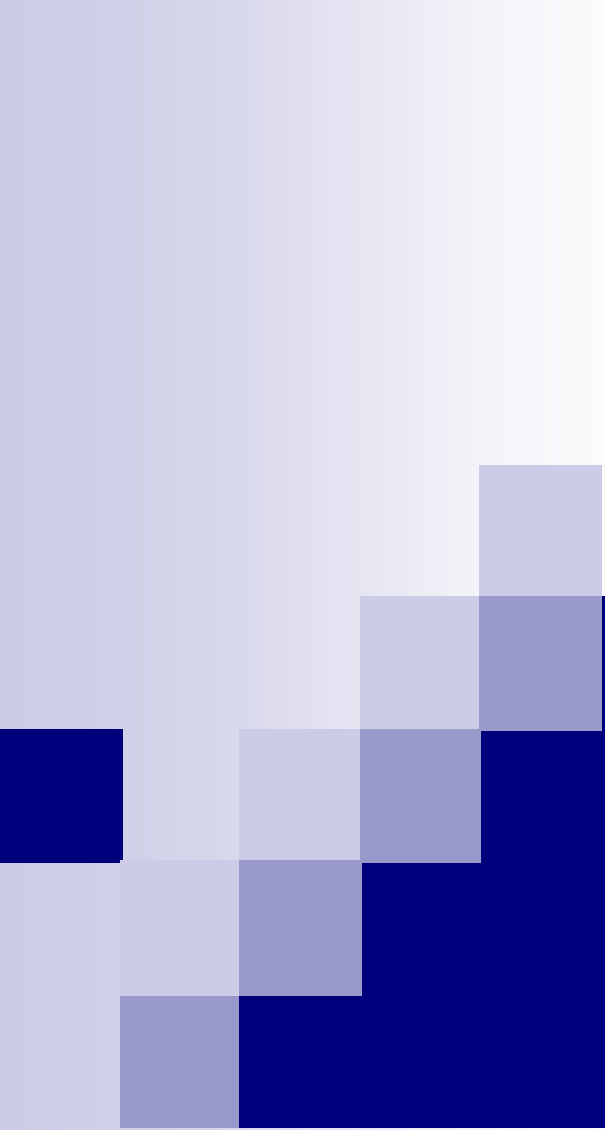
数值分析(7)

Numerical Analysis

计算机系 软件所 喻文健

第七章 数值积分与数值微分

- 积分: $I = \int_a^b f(x)dx$
- 微分: 用(若干)函数值近似计算其导数值
- 本章内容
 - 数值积分的基本概念
 - 牛顿-柯特斯公式
 - 复合求积方法
 - 理查森外推法
 - 自适应积分算法
 - 高斯求积公式
 - 数值微分



数值积分的基本概念

数值积分

$$\text{重心的横坐标} \bar{x} = \frac{\int_{x_1}^{x_2} x f(x) dx}{\int_{x_1}^{x_2} f(x) dx}$$

■ 目的与用途

□ 经典问题: 算几何形体的面积、
体积, 力学中物体的重心位置等

□ 例: 铝制波纹瓦的长度问题

由一块平整的铝板压制而成.

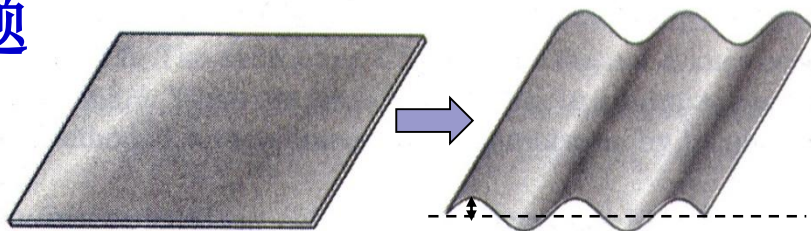
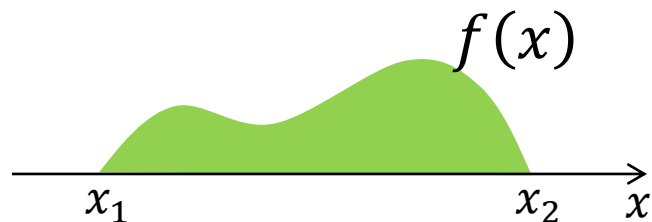
若每个波纹的高度(自中心线)

为1寸, 周期为 2π 寸, 做4尺长波纹瓦需多长铝板?

解: $f(x) = \sin x$, 求 $x=0$ 到 $x=40$ 寸之间的曲线弧长 L

$$L = \int_0^{40} \sqrt{1 + (f'(x))^2} dx = \int_0^{40} \sqrt{1 + (\cos x)^2} dx$$

第二类椭圆积分, 无法解析求出!



数值积分基本思想

■ 计算 $I(f) \triangleq \int_a^b f(x)dx$

□ **思路1**: Newton-Leibniz公式 $\int_a^b f(x)dx = F(b) - F(a)$,
其中 $F(x)$ 为 $f(x)$ 的**原函数**

□ 一些函数没有解析的原函数, 如 e^{x^2} , $\frac{\sin x}{x}$, $\sin x^2$ **或者被积函数表达式未知**

□ **思路2**: 积分的定义 $I(f) = \lim_{n \rightarrow \infty, h \rightarrow 0} \sum_{i=0}^n (x_{i+1} - x_i) f(\xi_i)$
这里 $h = \max_i (x_{i+1} - x_i)$

Monte Carlo
计算量大

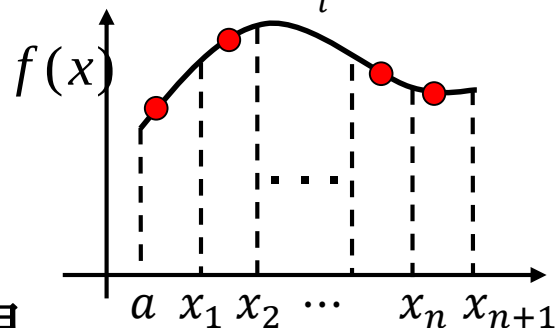
□ 取充分大的 n , 算函数值的加权和

□ **近似计算积分**的公式 (**机械求积公式**)

积分系数
积分节点

$$I_n(f) \triangleq \sum_{k=0}^n A_k f(x_k)$$

希望用较少的**计算量**得到较准确的结果

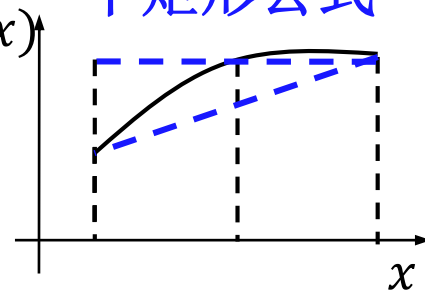


插值型求积公式

- 如何得到具体的求积公式? $I_n(f) = \sum_{k=0}^n A_k f(x_k)$
 - 用多项式函数 $p(x)$ 来近似 $f(x)$
 - $\int_a^b f(x)dx \approx \int_a^b p(x)dx$, 后者易于计算
 - **插值型求积公式**: 区间 $[a, b]$ 内插值节点为 $x_0, x_1, \dots, x_n$,
 $p(x) = \sum_{k=0}^n f(x_k)l_k(x) \longrightarrow I_n(f) = \sum_{k=0}^n f(x_k) \int_a^b l_k(x)dx$
其中 $l_k(x)$ 为拉格朗日插值基函数, 积分系数 $A_k = \int_a^b l_k(x)dx$
 - $n=0, n=1$ 对应的情况(积分节点取...)
 - $I_0(f) = \int_a^b f(x_0)dx = (b-a)f\left(\frac{a+b}{2}\right)$
 - $I_1(f) = \frac{b-a}{2}[f(a) + f(b)]$

梯形公式

中矩形公式



积分余项与代数精度

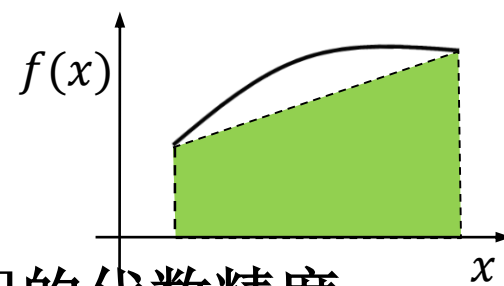
■ 积分余项

- **定义7.1:** 计算积分 $I(f)$ 的求积公式为 $I_n(f)$, 则 $R[f] \equiv I(f) - I_n(f)$ 为**积分余项** 反映计算的截断误差
- 若 $I_n(f)$ 为插值函数 $p(x)$ 的积分, $R[f] = \int_a^b [f(x) - p(x)] dx$ **插值余项的积分**
- 对插值型求积公式, $R[f] = \int_a^b \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x) dx$

■ 代数精度 衡量求积公式准确度的另一个指标

- **定义7.2:** 若 $I_n(f)$ 对于被积函数 $f(x)$ 为**次数不超过 m 的多项式**的情况都是准确的, 但对于 $m+1$ 次多项式不准确, 则该求积公式具有 m 次**代数精度** 注意: 有时候, 代数精度并非越高越好

积分余项与代数精度



■ 代数精度

- **例:** 梯形公式 $I_1(f) = \frac{b-a}{2} [f(a) + f(b)]$ 的代数精度
- 显然若 $f(x)$ 为 0 次、1 次多项式, 它准确. **为 1 次代数精度**
- 中矩形公式呢? $I_0(f) = (b-a)f\left(\frac{a+b}{2}\right)$

求积公式是线性表达式 □ **Th7.1:** 机械求积公式 $I_n(f) = \sum_{k=0}^n A_k f(x_k)$ 至少有 m 次代数精度 $\longleftrightarrow$ 当 $f(x)$ 分别为 $1, x, \dots, x^m$ 时, $I(f) = I_n(f)$

- **推论:** $\sum_{k=0}^n A_k = b-a$ (至少有 0 次代数精度)

- **Th7.2:** $I_n(f)$ 是插值型求积公式 $\longleftrightarrow$ 它至少有 n 次代数精度

- 证明: $\Rightarrow$, 插值型求积公式的定义

即插值型

- $\Leftarrow$, 令 $f(x) = l_k(x)$, 为 n 次多项式, 则 $A_k = \int_a^b l_k(x) dx$

积分余项与代数精度

推广的机械求积公式
(函数值、导数值的线性表达式)

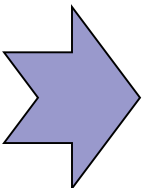
■ **例7.2:** 用公式 $H_2(f) = A_0 f(0) + A_1 f(1) + B_0 f'(0)$ 近似算积分 $I(f) = \int_0^1 f(x) dx$. 求系数值使该公式代数精度尽量高

□ 解: 令 $f(x) = 1, x, x^2$ 分别代入上述公式, 要使 $H_2(f) = I(f)$ 成立

□ 当 $f(x) = 1$ 时, $A_0 + A_1 = \int_0^1 1 \cdot dx = 1$

□ 当 $f(x) = x$ 时, $A_1 + B_0 = \int_0^1 x dx = \frac{1}{2}$

□ 当 $f(x) = x^2$ 时, $A_1 = \int_0^1 x^2 dx = \frac{1}{3}$


$$\begin{cases} A_0 = \frac{2}{3} \\ B_0 = \frac{1}{6} \end{cases}$$

□ 是2次代数精度吗? 还需考察 $f(x) = x^3$ 时公式是否准

□ 此时, $H_2(f) = A_1 \neq \int_0^1 x^3 dx$ ∴ 该公式为**2次代数精度**

求积公式的收敛性与稳定性

■ 收敛性

□ **定义7.3:** $I_n(f) = \sum_{k=0}^n A_k f(x_k)$, $a \leq x_0 < x_1 < \cdots < x_n \leq b$,
若 $\lim_{n \rightarrow \infty, h \rightarrow 0} I_n(f) = \int_a^b f(x) dx$, 其中 $h = \max_{1 \leq k \leq n} (x_k - x_{k-1})$,

称求积公式 $I_n(f)$ 具有**收敛性** (一系列求积公式的性质)

■ 数值积分问题的**敏感性分析**

$f(x) \rightarrow \tilde{f}(x)$, 扰动的大小 $\delta = \|f(x) - \tilde{f}(x)\|_{\infty} = \max_{a \leq x \leq b} |f(x) - \tilde{f}(x)|$

结果误差 $\left| \int_a^b f(x) dx - \int_a^b \tilde{f}(x) dx \right| \leq \int_a^b |f(x) - \tilde{f}(x)| dx \leq (b-a)\delta$

$\therefore (b-a)$ 是绝对**条件数**的上限 **敏感性取决于积分区间**

■ 数值稳定性

□ 考虑 $f(x_k) \rightarrow \tilde{f}(x_k)$, 则 $|I_n(f) - I_n(\tilde{f})| = ?$

求积公式的收敛性与稳定性

■ 数值稳定性(续)

敏感性: $|I - \tilde{I}| \leq (b-a)\delta$

□ 考察 $I_n(f)$ 的稳定性: $f(x_k) \rightarrow \tilde{f}(x_k)$

$$|I_n(f) - I_n(\tilde{f})| = \left| \sum_{k=0}^n A_k [f(x_k) - \tilde{f}(x_k)] \right| \leq \sum_{k=0}^n |A_k| \cdot |f(x_k) - \tilde{f}(x_k)| \\ \leq \left(\sum_{k=0}^n |A_k| \right) \varepsilon, \text{ 其中 } \varepsilon = \max_{0 \leq k \leq n} |f(x_k) - \tilde{f}(x_k)| \leq \delta$$

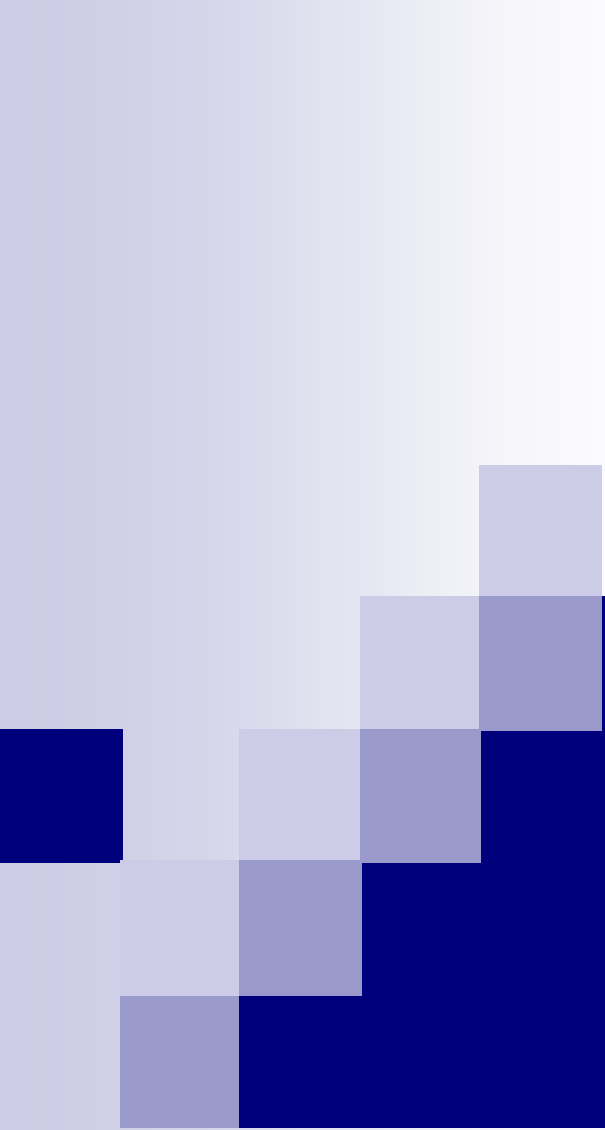
→ 若所有 $A_k > 0$, 则 $|I_n(f) - I_n(\tilde{f})| \leq \left(\sum_{k=0}^n A_k \right) \delta = (b-a)\delta$

这是控制数值计算误差能达到的理想情况

□ **定义7.4**: 若对 $k = 0, 1, \dots, n$, 均有 $A_k > 0$, 则机械求积公式 $I_n(f) = \sum_{k=0}^n A_k f(x_k)$ 是稳定的

尽量用稳定的公式

■ 对单个求积公式, 考察积分余项、代数精度和稳定性; 对一系列公式(节点逐渐增多), 考察收敛性
(估计截断误差)



牛顿-柯特斯公式

Newton-Cotes公式

■ 基本思想

- 插值型求积公式, 让节点 $x_k, k = 0, 1, \dots, n$ 在区间上均匀分布, 即设 $h = (b - a)/n$, 则 $x_k = a + kh, (k = 0, \dots, n)$

$$I_n(f) = \sum_{k=0}^n A_k f(x_k), \quad A_k = \int_a^b \frac{(x-x_0) \cdots (x-x_{k-1})(x-x_{k+1}) \cdots (x-x_n)}{(x_k-x_0) \cdots (x_k-x_{k-1})(x_k-x_{k+1}) \cdots (x_k-x_n)} dx$$

称为n阶牛顿-柯特斯公式

- 令 $x = a + th$, $A_k = \int_0^n \prod_{j=0, j \neq k}^n \left(\frac{t-j}{k-j} \right) \frac{b-a}{n} dt = (b-a) C_k^{(n)}$

- $C_k^{(n)}$ 与积分区间无关, 称为n阶N-C公式的Cotes系数

Cotes系数表	n=1,	1/2,	1/2	• 一系列求积公式 • 便于使用
	n=2,	1/6,	2/3, 1/6	
	n=3,	...	...	
	n=4,	7/90,	16/45, 2/15, 16/45, 7/90	
	...	...	...	

Newton-Cotes公式

Cotes系数表



■ 稳定性与常用公式

- $n=8$ 时, 有负的 $C_k^{(n)}$
- 相应的公式 **不稳定**

$$n=1, \quad 1/2, \quad 1/2$$

$n=2, \quad 1/6, \quad 2/3, \quad 1/6$

$$n=3, \quad \dots$$

n=4, 7/90, 16/45, 2/15, 16/45, 7/90

• • • • •

• • • • •

思考题 □ 对称性: $C_k^{(n)} = C_{n-k}^{(n)}$

$$n=8, \quad \frac{989}{28350} \quad \frac{5888}{28350} \quad \frac{-928}{28350} \quad \frac{10496}{28350} \quad \frac{-4540}{28350} \quad \frac{10496}{28350} \quad \frac{-928}{28350} \quad \frac{5888}{28350} \quad \frac{989}{28350}$$

- ### □ 三种常用的低阶N-C公式

$$n=1: T(f) = (b-a) \left[\frac{1}{2} f(a) + \frac{1}{2} f(b) \right]$$

梯形公式

n=2: $S(f) = (b-a) \left[\frac{1}{6} f(a) + \frac{4}{6} f\left(\frac{a+b}{2}\right) + \frac{1}{6} f(b) \right]$ **Simpson公式**

Simpson公式

$$n=4: \textcolor{blue}{C}(f) = (b-a) \left[\frac{7}{90} f(x_0) + \frac{32}{90} f(x_1) + \frac{12}{90} f(x_2) + \frac{32}{90} f(x_3) + \frac{7}{90} f(x_4) \right]$$

Cotes公式

中矩形公式可看成是 $n=0$ 时的特例

Newton-Cotes公式

■ 例7.3

- 用中矩形公式、梯形公式和辛普森公式近似计算积分

$$I(f) = \int_0^1 e^{-x^2} dx \quad (\text{准确值为 } I \approx 0.746824)$$

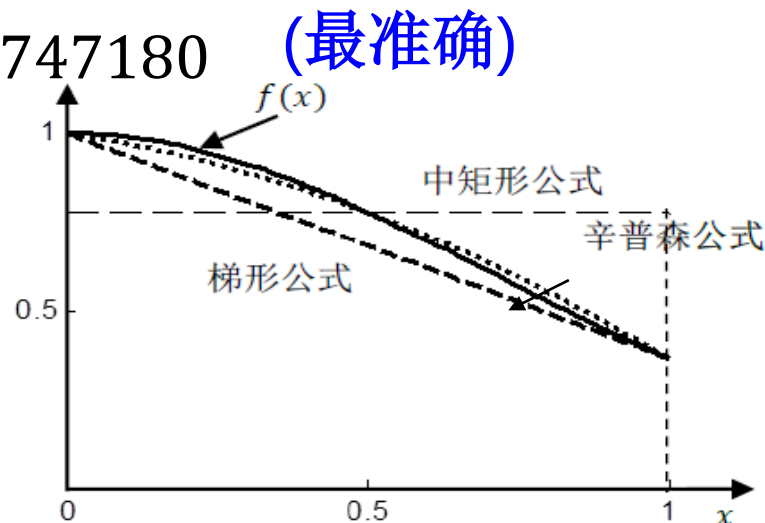
- $M = 1 \cdot e^{-0.25} \approx 0.778801$

- $T = \frac{1}{2}(1 + e^{-1}) \approx 0.683940$

- $S = \frac{1}{6}(1 + 4e^{-0.25} + e^{-1}) \approx 0.747180$ (最准确)

- 梯形法的误差约为-0.0629,
中矩形法的误差约为0.032

- 为什么?



Newton-Cotes公式

■ 代数精度 (n阶公式至少有n次代数精度)

□ **Th7.3**: 当n为偶数时, n阶牛顿-柯特斯公式 $I_n(f)$ 至少有 $n+1$ 次代数精度

□ **证明**: 只需看它对 $f(x) = x^{n+1}$ 的积分余项是否为0

n=0时
另外讨论

利用**插值型求积公式** $I_n(f)$ 的积分余项:

$$x=a+th, t \in [0, n]$$

$$R[f] = \int_a^b \frac{f^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x) dx = \int_a^b \omega_{n+1}(x) dx = h^{n+2} \int_0^n \prod_{j=0}^n (t-j) dt$$

关键看积分 $\int_0^n \prod_{j=0}^n (t-j) dt$, 令 $t = u + n/2$

n为偶数

$$\int_{-n/2}^{n/2} H(u) du, \text{ 其中 } H(u) = \prod_{j=0}^n \left(u + \frac{n}{2} - j\right) = \prod_{j=-n/2}^{n/2} (u-j)$$

∴一般不用
n=3对应的
N-C公式

n=2时, $H(u) = (u-1)u(u+1), \dots$

得证!

$H(u)$ 是**奇函数**! 在对称区间上的积分 $\int_{-n/2}^{n/2} H(u) du = 0$

低阶N-C公式的积分余项

类似Simpson公式余项,
可推出中矩形公式的余

■ 梯形公式(n=1)

项为 $\frac{f''(\eta)}{24}(b-a)^3$

$$\square R_T = I(f) - T(f) = \int_a^b \frac{f''(\xi)}{2!} (x-a)(x-b) dx$$

□ 第一积分中值定理: 函数 $f(x)$, $g(x)$ 有界可积, 且 $g(x)$ 不
改变正负号, $f(x)$ 连续, 则 $\int_a^b f(x)g(x)dx = f(\xi)\int_a^b g(x)dx$,

设原始 $f(x)$

充分光滑

$$\square R_T = \frac{f''(\eta)}{2} \int_a^b (x-a)(x-b) dx = -\frac{f''(\eta)}{12} (b-a)^3 \quad \xi \in (a, b)$$

■ Simpson公式(n=2)

变符号, 无法用积分中值定理

见例6.11
中的余项

$$\square R_S = I(f) - S(f) = \int_a^b \frac{f^{(4)}(\xi)}{4!} (x-a) \left(x - \frac{a+b}{2}\right)^2 (x-b) dx$$

□ 利用3次代数精度的性质推导

$$R_S = -\frac{f^{(4)}(\eta)}{2880} (b-a)^5, \quad \eta \in (a, b)$$

对在 $a, \frac{a+b}{2}, b$ 三点与 $f(x)$ 相
同、且在 $\frac{a+b}{2}$ 处与 $f'(x)$ 相同
的三次插值多项式 $H(x)$ 准确

稳定性、收敛性

■ 稳定性

$$n=8, \frac{989}{28350} \quad \frac{5888}{28350} \quad \frac{-928}{28350} \quad \frac{10496}{28350} \quad \frac{-4540}{28350} \quad \frac{10496}{28350} \quad \frac{-928}{28350} \quad \frac{5888}{28350} \quad \frac{989}{28350}$$

□ 前面指出, $n=8$ 的N-C公式不稳定

□ 当 $n \geq 10$ 时, $C_k^{(n)}$, ($k = 0, 1, \dots, n$)中至少有一个是负的

□ 高阶牛顿-柯特斯公式不稳定, 小扰动将带来大误差

■ 收敛性

□ 牛顿-柯特斯公式基于等距节点的多项式插值

□ 由于高次多项式插值的龙格现象, 随着 n 的增加, 并不能保证结果收敛到准确解

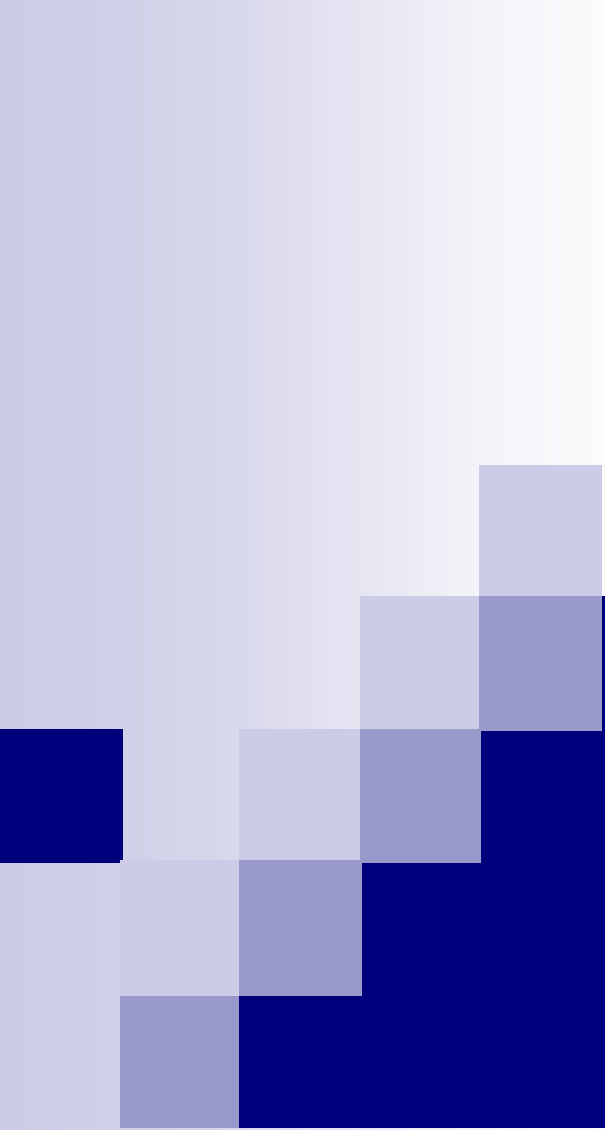
一般只用 $n < 8$ 的偶数阶

■ 计算量

□ n 阶公式含 $n+1$ 次函数求值

N-C公式

(梯形公式例外)



复合求积公式

复合求积公式

(composite quadrature)

■ 目的

- 高阶N-C公式不稳定, 阶数越高结果未必越准
- 受分段低次插值启发, 将积分区间分成小区间分别计算
- 例: 区间 $[a, b]$ 等分为 n 个子区间, 对每个采用梯形公式
积分误差: $\sum_{k=0}^{n-1} \frac{f''(\eta_k)}{12} \left(\frac{b-a}{n}\right)^3 \approx \frac{f''(\eta)(b-a)^3}{12n^2}$ n 增大, 误差减小

■ 复合梯形公式

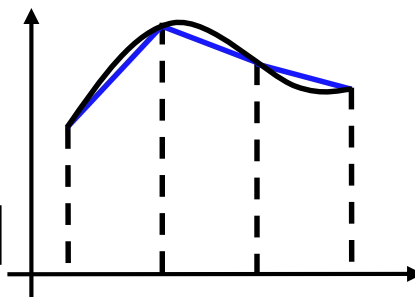
连续函数的中值定理

- 对区间做 n 等分, $h = (b - a)/n$

区间 $[x_k, x_{k+1}]$ 上, $\int_{x_k}^{x_{k+1}} f(x)dx \approx \frac{h}{2} [f(x_k) + f(x_{k+1})]$

$$T_n = \sum_{k=0}^{n-1} \frac{h}{2} [f(x_k) + f(x_{k+1})] = \frac{h}{2} [f(a) + 2 \sum_{k=1}^{n-1} f(x_k) + f(b)]$$

仍是“机械求积公式”！



复合求积公式

■ 复合梯形公式 (续)

- $T_n = \sum_{k=0}^{n-1} \frac{h}{2} [f(x_k) + f(x_{k+1})] = \frac{h}{2} [f(a) + 2 \sum_{k=1}^{n-1} f(x_k) + f(b)]$
- 是稳定的. 分段数 n 增大有收敛性 (分段线性插值的收敛性)
- **Th7.4:** $\lim_{n \rightarrow \infty} T_n = I(f)$ 只要 $f(x)$ 是可积函数就成立
- T_n 显然对于 $f(x) = x^2$ 不准确, $\therefore$ 只有1次代数精度
- 积分余项 2阶准确度
$$I(f) - T_n = - \sum_{k=0}^{n-1} \left[\frac{h^3}{12} f''(\eta_k) \right] = - \frac{h^2(b-a)}{12} f''(\eta) = O(h^2)$$
- **定义7.5:** 若等距节点求积公式的截断误差为 $O(h^p)$, h 为节点间距, 则它具有 p 阶准确度 (order of accuracy)
- 准确度阶数越高, 随 h 的减小其结果误差减小得越快

复合求积公式

■ 复合Simpson公式

- $S_n = \frac{h}{6} \sum_{k=0}^{n-1} [f(x_k) + 4f(x_{k+1/2}) + f(x_{k+1})]$

→ $S_n = \frac{h}{6} [f(a) + 4 \sum_{k=0}^{n-1} f(x_{k+1/2}) + 2 \sum_{k=1}^{n-1} f(x_k) + f(b)]$

- $h = x_{k+1} - x_k$, $x_{k+1/2}$ 为 $[x_k, x_{k+1}]$ 的中点

- S_n 公式中的 $[]$ 内, 部分项与复合梯形公式 T_n 的一样

- $2n + 1$ 个节点, 具有**稳定性**、**收敛性**、**3次代数精度**

- 积分余项 $I(f) - S_n = -\frac{1}{2880} h^4 (b - a) \cdot f^{(4)}(\eta) = O(h^4)$

- S_n 具有**4阶准确度** 与复合梯形公式对比, 看**例7.4**

还可以推导复合柯特斯公式 C_n

计算 $\int_0^1 \frac{\sin x}{x} dx$, 9 个点

复合求积公式

■ 步长折半的复合求积公式计算

- 积分余项公式含函数的高阶导数, 估计误差很麻烦. 常常**动态**地确定步长 h (逐渐减小, 直到满足要求)
- 常用的减小步长策略: **步长折半, 之前的计算可复用**
 n 个小区间 $\rightarrow$ $2n$ 个小区间, 记新增节点为 $x_{k+1/2}$

□ 复合梯形公式的情况

小区间 $[x_k, x_{k+1}]$, 步长折半后**复合梯形公式**结果为

$$\frac{h}{4}[f(x_k) + f(x_{k+1/2})] + \frac{h}{4}[f(x_{k+1/2}) + f(x_{k+1})]$$

递推化的复合梯形公式: $= \frac{1}{2} \left\{ \frac{h}{2} [f(x_k) + f(x_{k+1})] \right\} + \frac{h}{2} f(x_{k+1/2})$

$$T_{2n} = \frac{1}{2} T_n + \frac{h}{2} \sum_{k=0}^{n-1} f(x_{k+1/2}) \quad \text{只需再计算**新增节点**的函数值}$$

复合求积公式

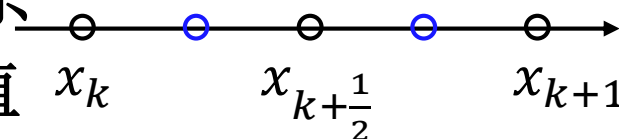
■ 步长折半的复合求积公式计算

□ 复合Simpson公式的情况

S_n 与 S_{2n} 的关系比复合梯形公式稍复杂

计算 S_{2n} 时仍可重用很多点的函数值

- 计算 S_n 的积分节点
- 计算 S_{2n} 的新增节点

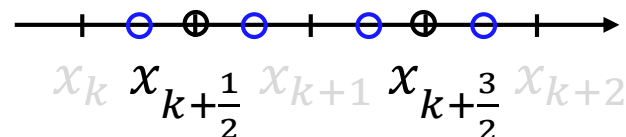


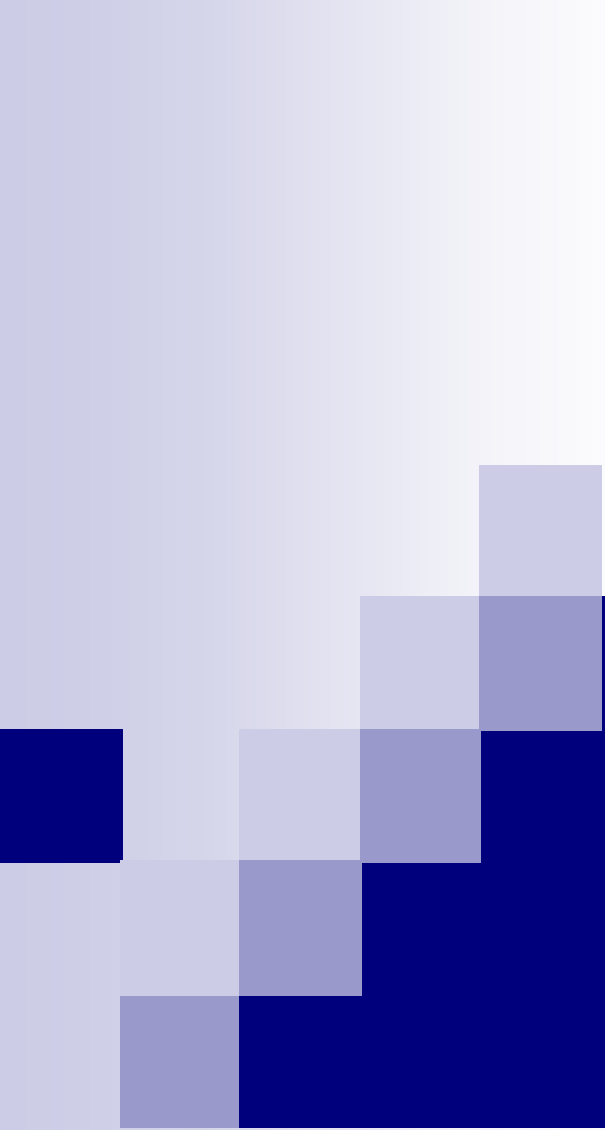
■ 很少使用**中矩形公式**的原因

$$M(f) = (b - a)f\left(\frac{a+b}{2}\right)$$

与梯形公式有**相同的**代数精度/准确度, **计算量更小**

但是, 在构造复合中矩形公式时, 计算量并不小,
且在步长折半时, **无法复用计算**





理查森外推方法

Romberg积分与理查森外推

- 复合梯形公式的余项 $I(f) - T_n = -\frac{h^2(b-a)}{12} f''(\eta) = O(h^2)$

Th7.5 □ 设被积函数 $f(x) \in C[a, b]$, 任意阶可导, 记 $T(h)$ 为积分步长为 h 的复合梯形公式的结果, 则 $T(h) = I(f) + \alpha_1 h^2 + \alpha_2 h^4 + \dots + \alpha_l h^{2l} + \dots$, 其中系数 $\alpha_l, (l = 1, 2, \dots)$ 与 h 无关

详见
课本

- 理查森外推法

(extrapolation)

- 区间逐次二等分; 再外推, 得到更高阶的求积公式

$$T_0^{(n)} - I = \alpha_1 h^2 + \alpha_2 h^4 + \dots$$

$$T_0^{(n+1)} - I = \frac{\alpha_1}{4} h^2 + \frac{\alpha_2}{16} h^4 + \dots$$

$$\xrightarrow{\quad} \frac{4T_0^{(n+1)} - T_0^{(n)}}{3} - I = \frac{4T_0^{(n+1)} - T_0^{(n)}}{3} - I = O(h^4)$$

更准确的值!

- 列三角形T数表计算

- 缺点: 光滑性要求高; 节点多

h	$T_0^{(n)}$	$T_1^{(n)}$	$T_2^{(n)}$
$b-a$	$T_0^{(0)}$		
$(b-a)/2$	$T_0^{(1)}$	$T_1^{(0)}$	
$\vdots$	$\vdots$	$\vdots$	$\vdots$

自适应积分算法

(本节内容不做考试要求)

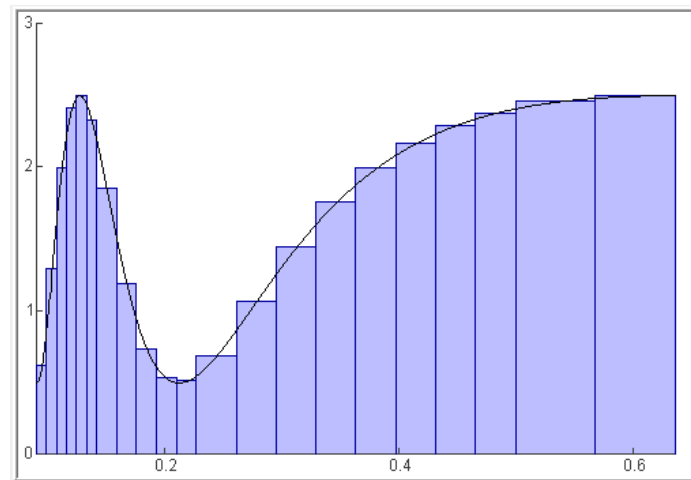


自适应积分算法 (adaptive quadrature)

■ 基本思想

例7.6

- Romberg算法效果不好的情况
- 积分节点没必要均匀分布
- 怎样自动地非均匀取点, 使计算结果达到准确度要求?
- 1. 评估当前区间积分结果的准确度, 若不准确就将它折半, 直至小区间的结果准确
- 2. 用两个不同的求积公式算同一个积分, 它们之差可近似判断结果的准确度



$$I - S \approx 2^4(I - S_2) \longrightarrow I - S_2 \approx \frac{S_2 - S}{15} \quad (\text{递归计算过程})$$

Simpson公式与复合Simpson公式

自适应积分算法

演示模块7.3, [quadgui](#)

■ 一个自适应求积算法

- 对每个区间, 用Simpson公式、复合Simpson公式计算
- 无论区间大小, 用相同的误差阈值; 函数的递归调用

□ **原理算法:** $Q = \text{ada_quad}(a, b, f(x), \text{tol})$

计算 $[a, b]$ 四等分后节点上的 $f(x)$ 值;

计算Simpson公式的值 S ;

计算复合Simpson公式的值 S_2 ;

If $|S_2 - S| < \text{tol}$

由 S, S_2 外推得到 Q ; $\{ Q = S_2 + \frac{S_2 - S}{15} \}$

Else

$Q_1 := \text{ada_quad}(a, (a+b)/2, f(x), \text{tol});$

$Q_2 := \text{ada_quad}((a+b)/2, b, f(x), \text{tol});$

$Q := Q_1 + Q_2;$

End

实际的算法需保证

函数值**不重复计算**

见课本的quadtx程序

自适应积分算法

■ 一个自适应求积程序quadtx

□ 例子: $I = \int_0^1 f(x) dx$

[Q, fcnt]= quadtx(@humps, 0, 1, 1e-2)

fcnt=41 (积分点数). 误差 4.1×10^{-4}

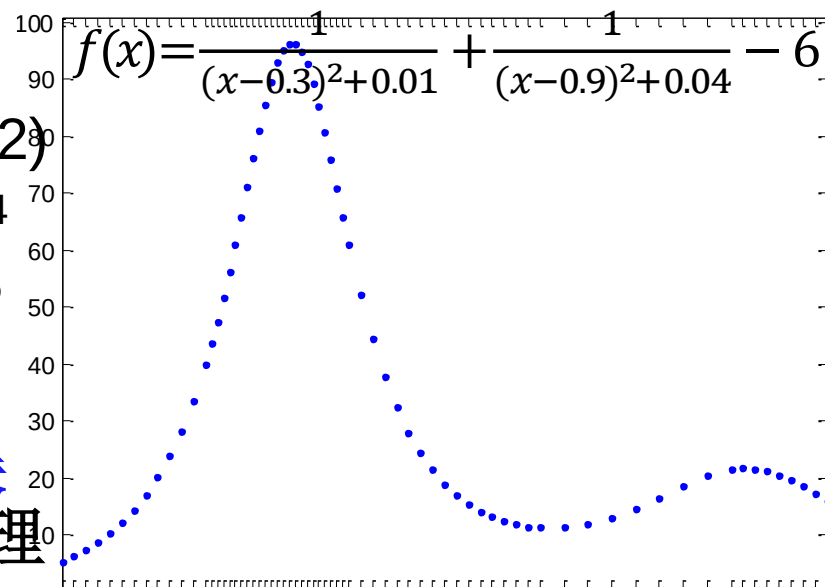
fcnt=53, 误差约 8×10^{-5}

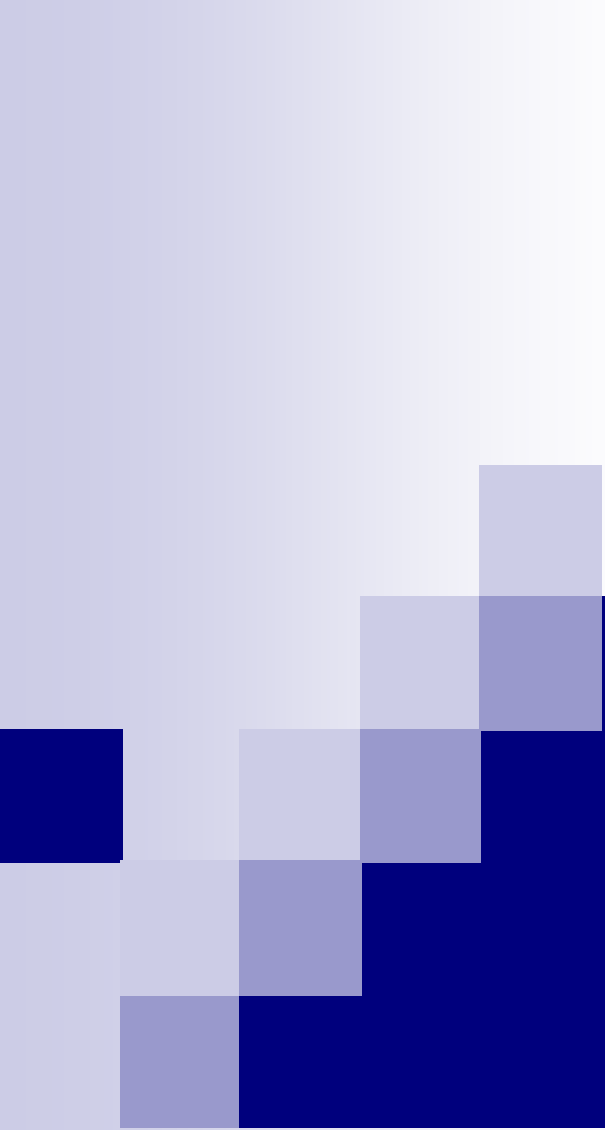
■ 更多讨论

□ 通过阈值设置控制相对误差; 不连续函数的特殊处理

□ 还有其他估计积分误差的方法, 比如利用中矩形公式和梯形公式的差

□ Romberg算法取33个、65个点分别算出29.8467, 29.8585 (误差: 0.012, 2×10^{-4}), 不如自适应积分好





高斯求积公式

高斯求积公式

■ 基本思想

- **牛顿-柯特斯公式**基于等距节点的多项式插值, 若不要求节点等间距, 可得更高代数精度的公式
- 求积公式 $I_n = \sum_{k=0}^n A_k f(x_k)$, 积分系数、积分节点一共是 $2n+2$ 个待定参数
- **定义7.6**: 若 $I_n = \sum_{k=0}^n A_k f(x_k)$ 具有 $2n+1$ 次或更高次代数精度, 则称 I_n 为 $(n$ 阶) **高斯求积公式**, 积分节点为 **高斯点** 一定是插值型公式
- **例7.8**: 推导计算 $\int_{-1}^1 f(x) dx$ 的高斯求积公式 $G_1(f) = A_0 f(x_0) + A_1 f(x_1)$
解得: $G_1(f) = f(-\frac{\sqrt{3}}{3}) + f(\frac{\sqrt{3}}{3})$
$$\begin{cases} A_0 + A_1 = \int_{-1}^1 1 dx = 2 \\ A_0 x_0 + A_1 x_1 = \int_{-1}^1 x dx = 0 \\ A_0 x_0^2 + A_1 x_1^2 = \int_{-1}^1 x^2 dx = \frac{2}{3} \\ A_0 x_0^3 + A_1 x_1^3 = \int_{-1}^1 x^3 dx = 0 \end{cases}$$

高斯求积公式

插值型求积公式、代数精度的概念也可扩展

■ 怎么求一般的高斯求积公式？

- 考虑带权积分 $I = \int_a^b f(x)\rho(x)dx \approx \sum_{k=0}^n A_k f(x_k) = I_n$
- 权函数为1、 $1/\sqrt{1-x^2}$ 、 e^{-x^2} 等, 节点/系数反映它的影响
- 若 $\forall f(x) \in \mathbb{P}_{2n+1}$, $I = I_n$, 则 I_n 为 $\rho(x)$ 对应的高斯求积公式
- **定理7.7**: $x_k, (k = 0, 1, \dots, n)$ 为高斯点的充要条件是多项式 $\omega_{n+1}(x) = (x-x_0)(x-x_1)\cdots(x-x_n)$ 与 $\forall P(x) \in \mathbb{P}_n$ 正交, 即

$$\int_a^b P(x)\omega_{n+1}(x)\rho(x)dx = 0$$

- 证明: $\longrightarrow$ 高斯积分有 $2n+1$ 次代数精度, 而 $\omega_{n+1}(x_k)=0$

要证 x_k 为节点的
公式有 $2n+1$ 次代
数精度

$\longleftarrow f(x) \in \mathbb{P}_{2n+1}$, 设 $f(x) = P_n(x)\omega_{n+1}(x) + Q_n(x) \longrightarrow \in \mathbb{P}_n$

$\int_a^b f(x)\rho(x)dx = \int_a^b Q_n(x)\rho(x)dx = \sum_{k=0}^n A_k Q_n(x_k)$ 根据插值型求积公式性质确定 A_k

$\therefore I_n$ 为高斯公式, x_k 为高斯点 $= \sum_{k=0}^n A_k f(x_k) = I_n$

高斯求积公式

■ 怎么求一般的高斯求积公式?

- Th7.7** □ x_k 为高斯点 $\longleftrightarrow \omega_{n+1}(x) = (x-x_0)(x-x_1)\cdots(x-x_n)$ 与 $\mathbb{P}_n$ 正交
- 高斯点就是 $n+1$ 次正交多项式的零点 正交多项式的性质(5)
 - 正交多项式与权函数有关, 且零点均为单实根, $\in (a, b)$
 - 求出高斯点后, 再根据插值型求积公式性质求系数

$$A_k = \int_a^b l_k(x) \rho(x) dx, \quad k = 0, \dots, n$$

■ 特点

$$= \sum_{j=0}^n A_j l_k^2(x_j) = A_k$$

- 稳定: $A_k = \int_a^b l_k^2(x) \rho(x) dx = \|l_k(x)\|_2^2 > 0$

- 收敛: $\lim_{n \rightarrow \infty} I_n = \int_a^b f(x) \rho(x) dx$ (Th7.9)

比自适应积分
使用更方便

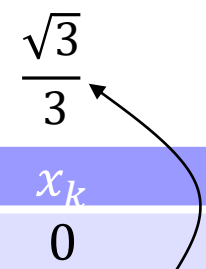
Th7.8 □ 积分余项: $R_n[f] = \frac{f^{(2n+2)}(\eta)}{(2n+2)!} \int_a^b \omega_{n+1}^2(x) \rho(x) dx$ (证明思路见课本)

高斯-勒让德公式

■ 高斯求积公式的应用

- 求正交多项式的零点, 再算积分系数→高斯积分表
- 勒让德多项式对应的区间为 $[-1, 1]$ 、权函数为 $\rho(x) = 1$
- 计算 $\int_{-1}^1 f(x)dx$ 的高斯积分公式称为高斯-勒让德公式
- 高斯-勒让德积分点关于0左右对称, 对称点的积分系数相同
- 高斯-切比雪夫公式、高斯-埃尔米特公式, 等等

高斯-勒让德积分表



n	x_k	A_k
0	0	2
1	± 0.5773503	1
2	± 0.7745967 0	0.5555556 0.8888889
3	± 0.8611363 ± 0.3399810	0.3478548 0.6521452
4	± 0.9061798 ± 0.5384693 0	0.2369269 0.4786287 0.5688889
5	± 0.9324695 ± 0.6612094 ± 0.2386192	0.1713245 0.3607616 0.4679139

高斯-勒让德公式

■ 例7.9

□ 用高斯-勒让德公式计算积分 $I = \int_0^1 \frac{\sin x}{x} dx$

□ 解: 为了变换积分区间, 设 $x = 0.5 + 0.5t$

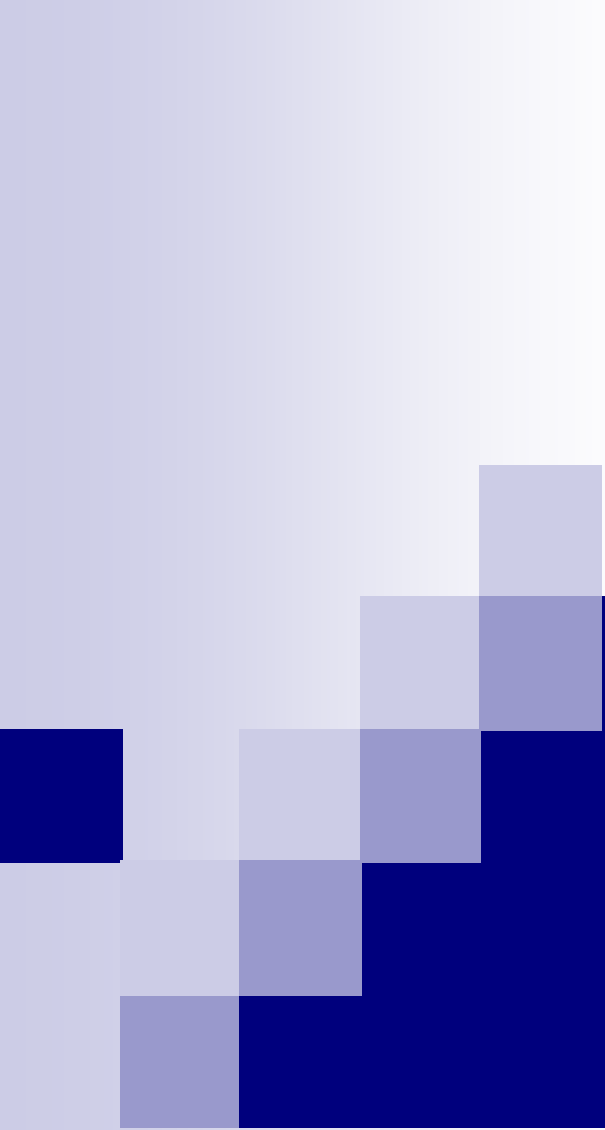
$$I = \frac{1}{2} \int_{-1}^1 \frac{\sin(0.5+0.5t)}{0.5+0.5t} dt \longrightarrow I_n = \sum_{k=0}^n A_k \frac{\sin(0.5+0.5x_k)}{0.5+0.5x_k}$$

□ 查n阶高斯-勒让德公式的积分节点与系数 x_k, A_k

□ 取 $n=4$ (5个节点), 算出 $\tilde{I} = I_n/2 = 0.94608312$

□ 在相同计算量情况下, 结果比Romberg算法更准(例7.5)

□ 与复合使用9个节点的Simpson公式结果差不多准,
(例7.4)
但计算量少



Matlab中的积分计算

(本节内容不做考试要求)

Matlab中的数值积分

$$\int_0^1 \frac{1}{\sqrt{1+x^4}} dx$$

■ 指定被积函数

□ 匿名函数@

```
>> f = @(x) 1./sqrt(1+x.^4)
```

□ 输入参数可以是向量，因此需采用逐项运算符号

□ M文件：可处理含奇异点的积分

```
Function f = sinc(r)
if x == 0
    f = 1;
else
    f = sin(x) ./ x;
end
```

$$\int_0^{\pi} \frac{\sin x}{x} dx$$

函数句柄@sinc

$$\beta(z, w) = \int_0^1 t^{z-1} (1-t)^{w-1} dt$$

□ 含多个自变量：(第1个是主自变量，其他为参数)

```
>> f_beta = @(t,z,w) t.^(z-1).*(1-t).^(w-1);
```

Matlab中的数值积分

■ 一维积分的命令

□ **quad** (第7.5节的自适应积分算法)

推荐

□ **integral/quadgk** (扩展的Gauss自适应积分, 见 § 7.6最后)

`[q, fcnt] = quad(fun, a, b, tol, trace, p1, p2,...)`

```
>> Q = quad(f, 0, 1)
>> quad (@sinc, 0, pi)    %function defined by M-file
ans =
    0.58949
>> [beta2_5, fcnt] = quad(f_beta, 0, 1, 1e-5, 0, 2, 5)
beta2_5 =
    0.033333
fcnt=
    17
```

积分参数

是否输出函数
计算次数
等. 0/非0

准确度控制:
缺省值 10^{-6}

Matlab中的数值积分

$$I = \int_a^b \int_{c(x)}^{d(x)} f(x, y) dy dx$$
$$I \approx \sum_{k=0}^n A_k \int_{c(x_k)}^{d(x_k)} f(x_k, y) dy$$

不用逐项运算符

■ 二重、三重积分

□ `integral2`, `quad2d`, `integral3`

■ 符号积分

□ 定义符号变量:

`syms`,

□ 符号积分: `int`

□ `simple` (表达式化简)

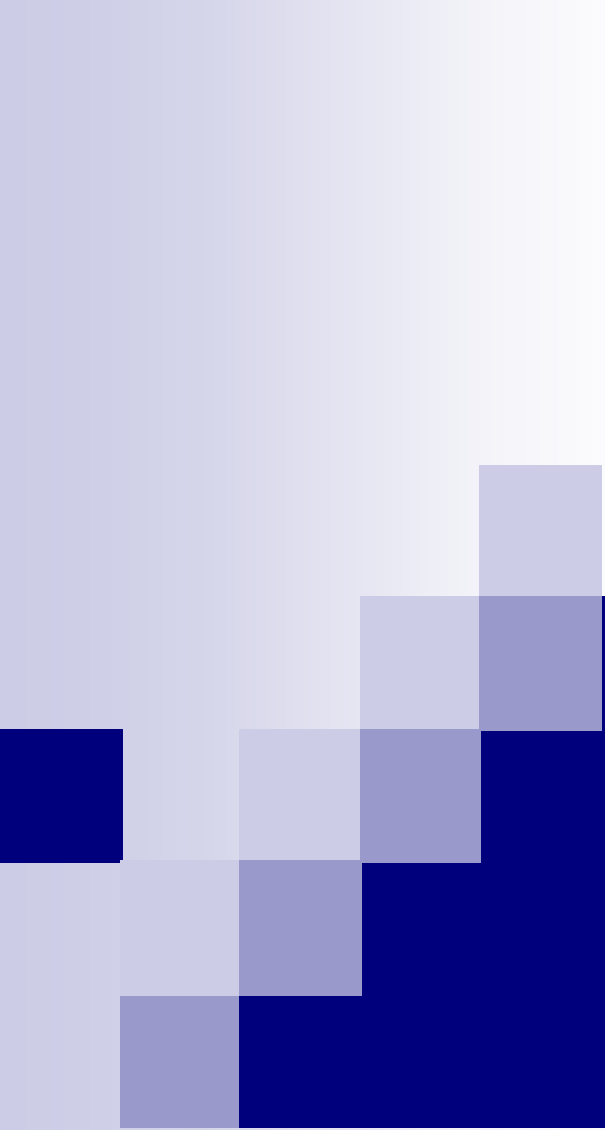
□ `double`

■ 离散数据点积分

□ 复合梯形法

□ `trapz(x, y)`

```
>> syms x
>> h = 1/((x-.3)^2+.01)+ 1/((x-.9)^2+.04) -6
h =
1/((x-3/10)^2+1/100)+1/((x-9/10)^2+1/25)-6
>> I = int(h)    %不定积分
I =
10*atan(10*x-3)+5*atan(5*x-9/2)-6*x
>> D = simple(int(h,0,1))    %定积分
D =
5*atan(16/13)-6+10*pi
>> Qexact = double(D)
Qexact =
29.858
```

数值微分

数值微分

■ 问题的描述

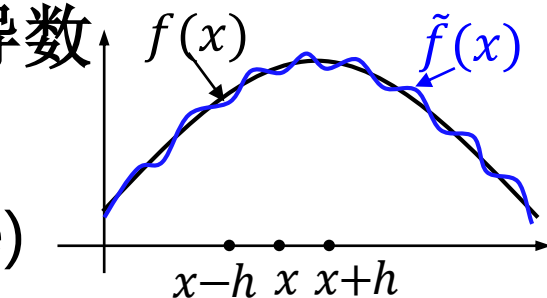
- 近似计算/表示函数导数 $f'(x)$, 其中 $f(x)$ 的表达式未知
- 使用若干函数值近似表示/计算其导数
- 与积分相比, 求微分的问题更敏感

■ 基本的有限差分公式(finite difference)

$$\square f'(x) \approx D_f(h) = \frac{f(x+h) - f(x)}{h} \quad (\text{向前差分})$$

$$\square f'(x) \approx D_b(h) = \frac{f(x) - f(x-h)}{h} \quad (\text{向后差分})$$

$$\square f'(x) \approx D_c(h) = \frac{f(x+h) - f(x-h)}{2h} \quad (\text{中心差分})$$



差商与导数的关系:

$$f[x_0, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}$$

2阶准确度

利用Taylor展开推出: $D_c(h) = f'(x) + \frac{f^{(3)}(\xi)}{3!} h^2 = f'(x) + O(h^2)$

数值微分

h	$D_c(h)$	准确数字位数
10^{-2}	0.3535544955	5
10^{-3}	0.3535534016	6
10^{-4}	0.3535533907	9
10^{-5}	0.3535533906	9
10^{-6}	0.3535533906	9
10^{-7}	0.3535533899	7
10^{-8}	0.3535533977	8
10^{-9}	0.3535534088	6
10^{-10}	0.3535538529	6
10^{-11}	0.3535505222	5
10^{-12}	0.3536060333	3

■ 基本的有限差分公式 (续)

□ **例7.10:** 用中心差分公式算 $f(x) = \sqrt{x}$ 在 $x=2$ 处一阶导数值, 分析不同步长 h 对准确度影响 (截断误差、舍入误差)

□ 解: 中心差分公式 $D_c(h) = \frac{f(2+h) - f(2-h)}{2h} = \frac{\sqrt{2+h} - \sqrt{2-h}}{2h}$

取 $h=10^{-1}, 10^{-2}, 10^{-3}, \dots, 10^{-12}$, 计算结果见表7-8

准确导数值: $f'(2) = \sqrt{2}/4$, 随 h 缩小, 误差先减后增?

分析: 误差限 $\varepsilon_{tot} = \frac{M \cdot h^2}{6} + \frac{\varepsilon}{h}$, M 为 $|f'''(\xi)|$ 上界, ε 为算 $f(x)$

总误差限何时最小? 的误差限

$\frac{M \cdot h^2}{6} = \frac{\varepsilon}{2h}$, 即 $h = \sqrt[3]{\frac{3\varepsilon}{M}}$ 时, 达最小值

基本符合实验结果!

$|f'''(\xi)| = \frac{3}{8} \xi^{-5/2} \leq 0.0754$, $\varepsilon \approx 2 \times 10^{-16}$ $\Rightarrow h \approx 2 \times 10^{-5}$ 时误差最小

数值微分

$$f[x_0, \dots, x_n] = \frac{f^{(n)}(\xi)}{n!}$$

■ 基本的有限差分公式 (续)

□ 二阶中心差分: $f''(x) \approx G_c(h) = 2f[x-h, x, x+h]$

$$G_c(h) = f''(x) + \frac{f^{(4)}(\xi)}{12} h^2 = f''(x) + O(h^2) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$$

■ 插值型求导公式 (上述公式均为此方法的特例)

□ 根据离散点上 $f(x)$ 值构造插值多项式 $P(x)$, 用它的导数近似 $f(x)$ 的导数 (适合于节点任意分布的情况)

$$L_n(x) = \sum_{k=0}^n f(x_k) l_k(x) \approx f(x) \xrightarrow{\text{green arrow}} f^{(i)}(x) \approx \sum_{k=0}^n f(x_k) l_k^{(i)}(x)$$

□ 增加插值节点可构造出更高准确度的公式, 或得到求更高阶导数的差分公式

带入 x_j

详见课本7.7.2节的推导

数值微分

■ 外推算法

□ 中心差分公式的误差展开式

□ 将 h 逐次减半, 用理查森外推构造更准的公式

$$f'(x) - D_c(h) = \frac{\alpha_1}{4} h^2 + \frac{\alpha_2}{16} h^4 + \dots \longrightarrow f'(x) - \frac{4D_c\left(\frac{h}{2}\right) - D_c(h)}{3} = O(h^4)$$

$\searrow D_c^{(1)}(h)$

□ 可构造逐次外推的求导算法
(考虑舍入、计算量, 外推次数别太多)

□ 例7.11

$$f(x) = x^2 e^{-x}$$

算 $x = 0.5$ 处的导数

准确的有效
数字位数

h	$D_c(h)$	$D_c^{(1)}(h)$	$D_c^{(2)}(h)$
0.1	0.4516049081		
0.05	0.4540761694	0.4548999231	
0.025	0.4546926288	0.4548981152	0.4548979947
p	3	5	9

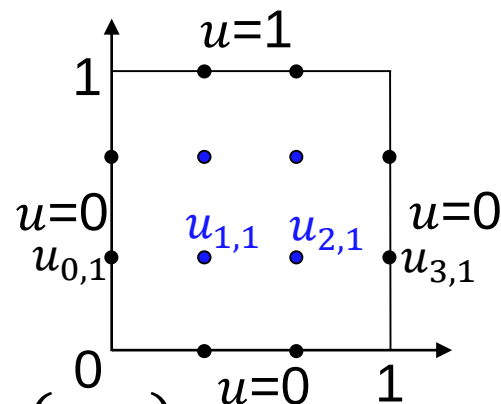
数值微分的应用

■ 有限差分法解偏微分方程(第4.4.1节)

□ 正方形区域的二维Laplace方程

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0, \quad x, y \in (0, 1)$$

□ 已知 $u(x, y)$ 在边界上的值, 求区域内的 $u(x, y)$.



每边均匀分 $n+1$ 份得二维离散网格, 在网格点上设变量 $u_{i,j}$

二阶中心差分 $\frac{\partial^2 u(x_i, y_j)}{\partial x^2} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}$ 代入微分方程, 对每个格点得一个代数方程

例如 $n=2$

$$\begin{cases} 4u_{1,1} - u_{0,1} - u_{2,1} - u_{1,0} - u_{1,2} = 0 \\ 4u_{2,1} - u_{1,1} - u_{3,1} - u_{2,0} - u_{2,2} = 0 \\ 4u_{1,2} - u_{0,2} - u_{2,2} - u_{1,1} - u_{1,3} = 0 \\ 4u_{2,2} - u_{1,2} - u_{3,2} - u_{2,1} - u_{2,3} = 0 \end{cases} \longrightarrow \begin{bmatrix} 4 & -1 & -1 & 0 \\ -1 & 4 & 0 & -1 \\ -1 & 0 & 4 & -1 \\ 0 & -1 & -1 & 4 \end{bmatrix} \begin{bmatrix} u_{1,1} \\ u_{2,1} \\ u_{1,2} \\ u_{2,2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$